Self-Organization of Robots in a Hostile Environment

Multi-Agent System Project Report

Adnane Erekraken • Mounia Abdelmoumni • Daniel Fernàndez De La Mela

MSc AI — CentraleSupélec

April 2026

1. Introduction and Motivation

This report presents the design, implementation, and evaluation of a multi-agent system (MAS) for autonomous robotic waste cleanup in a hostile environment, developed as part of the Multi-Agent Systems course at CentraleSupélec (Université Paris-Saclay). The simulation models a realistic scenario where heterogeneous robots must collaboratively collect, transform, and dispose of radioactive waste across a spatially partitioned environment.

The project is structured around three progressively sophisticated implementation steps: agents without communication (Step 1), agents with communication (Step 2), and agents with orchestration and uncertainty handling (Step 3). Each step builds upon the previous, enabling systematic evaluation of how coordination mechanisms influence mission performance.

Project Objective

Design and simulate a decentralized multi-robot system capable of 100% waste disposal completion across diverse configurations, with measurable efficiency gains at each coordination tier. Evaluate the impact of communication, orchestration, and uncertainty-aware scoring on completion rate, speed, and remaining waste.

2. Environment and Agent Design

2.1 Environment Structure

The simulation environment is a discrete MultiGrid partitioned into three horizontal zones (z1, z2, z3) ordered from west to east. Each zone hosts a Radioactivity passive agent whose level is drawn uniformly at random within the zone's prescribed interval:

Zone	Radioactivity Range	Role
z1 — Low	*[0.00, 0.33)*	*Green waste spawning zone; green robots only*
z2 — Medium	*[0.33, 0.66)*	*Intermediate handoff zone*
z3 — High	*[0.66, 1.00]*	*Final disposal zone; red robots only*

Table 1 — Zone definitions and their ecological roles.

A WasteDisposal agent is placed at a randomly selected y-position in the easternmost column of z3. The disposal cell's location is initially unknown to robots and must be discovered through exploration or inter-agent communication.

2.2 Waste Types and Robot Capabilities

Three waste types and three robot types form the pipeline, encoding a staged transformation process analogous to a distributed manufacturing chain:

Robot Type	Zone Access	Primary Role
GreenAgent	*z1 only*	*Collect 2× green → transform to 1× yellow; drop yellow at z1/z2 boundary*
YellowAgent	*z1, z2*	*Collect 2× yellow → transform to 1× red; drop red at z2/z3 boundary*
RedAgent	*z1, z2, z3*	*Collect 1× red → transport to disposal cell; permanently dispose of waste*

Table 2 — Robot type capabilities and zone constraints.

2.3 Perception–Deliberation–Action Loop

All agents implement the standard MAS loop prescribed by the assignment:

```
percepts = model.get_percepts(self)  →  _update_knowledge(percepts)  →
action = deliberate(knowledge)  →  model.do(self, action)  →
_update_knowledge(new_percepts)
```

Perception uses a Von Neumann neighborhood (current cell plus four orthogonal neighbors). The deliberate() function is strictly isolated: it may only access self.knowledge, respecting the assignment constraint. The do() method in model.py validates feasibility before applying any state transition.

2.4 Knowledge Representation

Each robot maintains a stateful knowledge base. Key fields include: current position and zone, current percepts, inventory (InventoryWaste tokens rather than live Mesa agents), discovered zone map, known waste locations, visited cell set, disposal location when found, current navigation target, and last-read message step.

> **Design Choice — InventoryWaste Tokens**
>
> *Carrying live Mesa Waste agents in robot inventories caused visualization crashes because Mesa's scheduler assumes all registered agents have a valid grid position. Replacing carried items with lightweight InventoryWaste tokens at pickup, and re-instantiating full Waste agents on drop, cleanly separates the grid state from the agent inventory state.*

3. Communication Design (Step 2)

3.1 Shared Message Board

Step 2 equips each agent with reading/writing access to a model-level message board (self.message_board). Messages are broadcast asynchronously and expire after a TTL of 50 simulation steps, preventing stale information from polluting agent beliefs.

3.2 Message Taxonomy

Message Type	Semantic Meaning
waste_picked	*Sender has removed waste from a cell; others should invalidate their known-waste cache for that position*
waste_dropped	*Sender has deposited waste at a cell; others can update their target queue*
disposal_found	*Sender has discovered the disposal cell location; all robots can now navigate directly*

Table 3 — Message types and their effect on receiving agents' knowledge bases.

3.3 Rationale

The blackboard pattern avoids direct agent-to-agent coupling and maps naturally onto the implicit coordination paradigm discussed in the course. Agents still deliberate from their own knowledge; communication enriches, rather than replaces, local belief. TTL-based expiry handles the case where a robot reports a waste location that is subsequently removed by another agent before the message expires.

4. Step 3 — Orchestration and Uncertainty-Aware Scoring

4.1 Mission Orchestrator

Step 3 introduces a centralized MissionOrchestrator at the model level. At each simulation tick, the orchestrator computes short-horizon target assignments for all active robots, reducing duplicate chasing and optimizing global throughput. Crucially, the orchestrator does not replace agent deliberation: assignments are injected as guidance targets that agents combine with their local percepts, memory, and message-board updates.

4.2 Uncertainty-Aware Scoring Function

The default nearest-only assignment was augmented with a weighted scoring function that balances multiple objectives:

```
score = Manhattan_distance + α·radioactivity + β·crowding −
γ_east·eastward_bonus − δ·scarcity_bonus
```

Calibrated weights: *radioactivity penalty $\alpha = 2.967$, crowding penalty $\beta = 1.695$, eastward bonus for regular robots $\gamma = 0.567$, for red robots $\gamma = 0.414$, scarcity bonus (≤2 items) $\delta = 0.885$, scarcity bonus (=1 item) $\delta = 2.074$.*

4.3 Weight Calibration

Weights were searched over a bounded grid with constraint scarcity_eq1_bonus ≥ scarcity_le2_bonus. The calibration objective penalizes incompletion heavily, then residual waste, then total steps, then message volume:

```
objective = (1 - completion_rate)×1000 + remaining_mean×25 + steps_mean +
messages_mean/1000
```

The priority ordering ensures that speed gains are never traded for completion reliability, a critical property for safety-critical cleanup missions.

4.4 Supplementary Coordination Rules

Several engineering fixes address failure modes discovered empirically:

- *Rendezvous strategy for Green/Yellow robots stuck with one unpaired item: robot moves to a shared meeting point and drops for consolidation by another robot of the same type.*
- *Deterministic conflict resolution at rendezvous cells: same-type robot with the lowest unique_id picks first, eliminating non-deterministic contention.*
- *Red robot singleton fallback: a RedAgent collects yellow or green waste only when exactly one item of that color remains in the entire system, preventing interference with the normal transformation pipeline.*
- *Anti-repickup cooldown: a short post-drop delay prevents immediate pick-drop loops.*

5. Experimental Results

All experiments were conducted with 20 seeds (base seed 100), 500 maximum steps per run, and four benchmark configurations designed to stress-test different failure modes: balanced small (C1), green-robot-heavy (C2), pipeline-strong (C3), and red-robot-limited (C4).

5.1 Configuration Descriptions

Configuration	Parameters	Design Intent
C1 — Balanced Small	*n_green=3, n_yellow=2, n_red=2, n_wastes=15*	*Baseline balanced setup*
C2 — Green Heavy	*n_green=5, n_yellow=2, n_red=2, n_wastes=25*	*Supply-side bottleneck at z1*
C3 — Pipeline Strong	*n_green=4, n_yellow=3, n_red=3, n_wastes=25*	*Well-staffed full pipeline*
C4 — Red Limited	*n_green=4, n_yellow=2, n_red=1, n_wastes=20*	*Disposal bottleneck at z3*

Table 4 — Benchmark configuration descriptions.

5.2 C1 — Balanced Small

5.3 C2 — Green Heavy

Mode	Completion	Steps (mean ± std)	Remaining (mean ± std)	Messages (mean ± std)
Step 1 — No Communication	*100.0%*	*240.65 ± 69.00*	*0.00 ± 0.00*	*0*
Step 2 — Communication	*100.0%*	*111.85 ± 22.23*	*0.00 ± 0.00*	*119.65 ± 16.87*
Step 3 — Nearest-only	*100.0%*	*106.05 ± 14.47*	*0.00 ± 0.00*	*123.00 ± 12.00*
Step 3 — Uncertainty-aware	*100.0%*	*110.60 ± 24.90*	*0.00 ± 0.00*	*126.00 ± 19.03*

Table 6 — C2 results (20 seeds).

C2 Analysis

C2 has 5 green robots collecting 25 wastes, producing a large yellow handoff queue. Communication (Step 2) provides a strong speedup: −128 steps vs Step 1 (240.65 → 111.85), because waste_dropped messages efficiently direct yellow robots to the z1/z2 boundary without redundant exploration. Step 3 nearest-only further reduces steps by ~6 (111.85 → 106.05) with lower variance (±14.47 vs ±22.23). Unexpectedly, uncertainty-aware scoring underperforms nearest-only in C2 (+4.55 steps, +3.00 messages). The scarcity and eastward bonuses add overhead in this supply-rich scenario where direct nearest-target navigation is already near-optimal.

5.4 C3 — Pipeline Strong

Mode	Completion	Steps (mean ± std)	Remaining (mean ± std)	Messages (mean ± std)
Step 1 — No Communication	*100.0%*	*158.20 ± 57.84*	*0.00 ± 0.00*	*0*
Step 2 — Communication	*100.0%*	*95.80 ± 18.06*	*0.00 ± 0.00*	*111.35 ± 16.34*
Step 3 — Nearest-only	*100.0%*	*84.90 ± 10.91*	*0.00 ± 0.00*	*106.15 ± 8.73*
Step 3 — Uncertainty-aware	*100.0%*	*91.00 ± 19.12*	*0.00 ± 0.00*	*111.15 ± 10.59*

Table 7 — C3 results (20 seeds).

C3 Analysis

C3 is the best-balanced configuration with adequate robots at every stage. All three modes achieve 100% completion. The step progression (158.20 → 95.80 → 84.90) demonstrates clean additive benefits: communication reduces inter-zone search, and nearest-only orchestration further eliminates redundant motion. Nearest-only achieves the best performance here (84.90 steps, ±10.91). Uncertainty-aware scoring again trails nearest-only (+6.10 steps) — the crowding

and radioactivity penalties appear to cause suboptimal detours in a well-staffed environment where robots are rarely in genuine conflict. Note that variance drops markedly from Step 1 (±57.84) to Step 3 nearest (±10.91), confirming that orchestration stabilizes trajectories.

5.5 C4 — Red Limited

Mode	Completion	Steps (mean ± std)	Remaining (mean ± std)	Messages (mean ± std)
Step 1 — No Communication	85.0%	311.65 ± 129.16	0.20 ± 0.52	0
Step 2 — Communication	80.0%	244.80 ± 157.81	0.50 ± 1.10	186.30 ± 145.24
Step 3 — Nearest-only	100.0%	121.45 ± 39.70	0.00 ± 0.00	73.75 ± 6.29
Step 3 — Uncertainty-aware	100.0%	113.35 ± 27.28	0.00 ± 0.00	74.45 ± 6.64

Table 8 — C4 results (20 seeds).

C4 Analysis

C4 is the most discriminating configuration: with only 1 red robot and 20 wastes, the disposal bottleneck stresses coordination. Step 1 and Step 2 both fail to reach 100% completion (85% and 80% respectively). The Step 2 regression (85% → 80%) is explained by high-variance coordination conflicts in a bottleneck scenario where the single red robot is the critical path. Step 3 resolves this entirely, both variants achieve 100% completion with dramatically reduced mean steps (~2.7× speedup vs Step 1). Uncertainty-aware scoring outperforms nearest-only in C4 (113.35 vs 121.45 steps, −8.10), with substantially lower variance (±27.28 vs ±39.70). The scarcity bonus effectively guides the red robot toward the last remaining items, reducing idle search.

6. Cross-Configuration Analysis

6.1 Completion Rate Summary

Mode	C1	C2	C3	C4
Step 1 — No Communication	100%	100%	100%	85%
Step 2 — Communication	95%	100%	100%	80%
Step 3 — Nearest-only	100%	100%	100%	100%
Step 3 — Uncertainty-aware	100%	100%	100%	100%

Table 9 — Completion rates across all configurations and modes.

The most salient finding is that Step 3 (both variants) achieves 100% completion across all four configurations, while Steps 1 and 2 both exhibit failures in at least one configuration. This robustness is the primary contribution of the orchestration layer.

6.2 Mean Steps Comparison

Mode	C1	C2	C3	C4
Step 1 — No Communication	*183.10*	*240.65*	*158.20*	*311.65*
Step 2 — Communication	*117.40*	*111.85*	*95.80*	*244.80*
Step 3 — Nearest-only	*88.05*	*106.05*	*84.90*	*121.45*
Step 3 — Uncertainty-aware	*86.30*	*110.60*	*91.00*	*113.35*

Table 10 — Mean completion steps (lower is better). Cells in bold row correspond to best performer per column.

Averaged across all four configurations, uncertainty-aware scoring achieves the lowest mean steps (100.31), narrowly beating nearest-only (100.11, nearly tied). Both Step 3 variants are dramatically faster than Steps 1 and 2. Communication (Step 2) provides the largest single improvement for supply-rich configurations (C2: −128 steps), while orchestration (Step 3) provides the largest improvement for constrained configurations (C4: −131 steps for nearest-only vs Step 2).

6.3 Variance Reduction

A consistent pattern across configurations: orchestration reduces variance substantially. Step 1 variance in C4 reaches ±129 steps; Step 3 nearest-only reduces this to ±39.70, and uncertainty-aware to ±27.28. This stabilization is critical for deployment reliability, high variance implies that some seeds take dramatically longer than average, potentially exceeding real-world time budgets.

6.4 Communication Cost

Step 3 generates fewer messages than Step 2 in C4 (74 vs 186) because the orchestrator eliminates redundant exploratory behavior. In C1–C3, message counts are comparable between Steps 2 and 3. The TTL expiry mechanism effectively caps message board size, preventing unbounded memory growth in long-running simulations.

6.5 Nearest-Only vs Uncertainty-Aware: When Each Wins

Configuration	Winner	Delta (uncertainty-aware vs nearest-only)
C1 — Balanced Small	*Uncertainty-aware*	*−1.75 steps, −1.40 messages*
C2 — Green Heavy	*Nearest-only*	*−4.55 steps, −3.00 messages*

Configuration	Winner	Delta (uncertainty-aware vs nearest-only)
C3 — Pipeline Strong	*Nearest-only*	*−6.10 steps, −4.50 messages*
C4 — Red Limited	*Uncertainty-aware*	*−8.10 steps, −0.70 messages*

Table 11 — Head-to-head comparison of Step 3 variants.

Uncertainty-aware scoring wins in C1 and C4, scenarios where resource scarcity or bottleneck agents benefit from explicit scarcity bonuses. Nearest-only wins in C2 and C3, scenarios where supply is abundant and extra penalty terms introduce navigation overhead. This suggests that the optimal scoring strategy is environment-dependent, and a meta-selector that switches between strategies based on detected resource scarcity could outperform either fixed strategy.

7. Architecture and Design Discussion

7.1 Discrete Grid World

A discrete MultiGrid was chosen because the assignment is fundamentally spatial, and local perception is central to the agent design. Continuous-space alternatives (e.g., velocity-based navigation) would add unnecessary complexity without adding fidelity to the core MAS research question.

7.2 Passive Object Agents

Radioactivity, Waste, and WasteDisposal are implemented as agents without step() behavior. This design keeps the environment explicit and easily introspectable, and integrates naturally with Mesa's agent scheduler while avoiding the need for a separate environment data structure.

7.3 Environment-Side Action Validation

The `do()` *method in model.py validates feasibility before applying any state transition. This separation between agent intention and environment state transition is a best-practice in MAS design: agents may have incorrect beliefs about the feasibility of their chosen actions, and the environment must remain the ground-truth authority.*

7.4 Thread-Safe Data Collection

A SafeDataCollector wrapper with a threading.Lock was added around Mesa's DataCollector to prevent intermittent race conditions under Solara's reactive rendering model, which runs visualization updates concurrently with simulation ticks.

7.5 Limitations

- *Navigation is greedy (Manhattan distance heuristic), not globally optimal. A full path planner (e.g., A*) would handle environments with obstacles, but is unnecessary for the open-grid assignment scenario.*
- *No collision model: multiple robots can occupy the same cell simultaneously. Collision avoidance would be needed for a physical deployment.*

- *The orchestrator uses a centralized model-level assignment, which is technically omniscient about global waste positions. In a fully decentralized deployment, this information would need to be distributed via communication.*
- *Statistical significance tests are included in the benchmark scripts but only for selected pairwise comparisons; a full factorial ANOVA is not yet reported.*

8. Conclusions

This project successfully implements all three steps of the MAS assignment, demonstrating a clear performance hierarchy:

- *Step 1 (no communication) achieves 100% completion in balanced configurations but fails in constrained scenarios (C4: 85%).*
- *Step 2 (communication) dramatically reduces mean steps in supply-rich configurations but can reduce completion reliability due to coordination conflicts in bottleneck scenarios.*
- *Step 3 (orchestration) achieves 100% completion in all configurations, reduces variance, and delivers the fastest mean completion times across the benchmark suite.*

The calibrated uncertainty-aware scoring function outperforms nearest-only assignment in configurations where resource scarcity is the binding constraint (C1, C4), while nearest-only is preferable in well-supplied configurations (C2, C3). This finding motivates future work on adaptive meta-strategies that dynamically switch scoring modes based on environment state monitoring.

Key Takeaway

Orchestration with deterministic conflict resolution and singleton fallback rules is necessary and sufficient for 100% completion robustness across all tested configurations. Communication alone is beneficial on average but insufficient for reliability under bottleneck conditions. The uncertainty-aware scoring provides marginal but consistent gains when resource scarcity is present, and should be preferred as the default in deployment scenarios where the robot count is uncertain or potentially insufficient.

Appendix — Calibrated Uncertainty Weights

The following weights were selected from the calibration search over 20 seeds per candidate, minimizing the composite objective function described in Section 4.3:

Parameter	Value	Role
radio_penalty_weight	*2.967*	*Penalizes routes through high-radioactivity cells*
crowd_penalty_weight	*1.695*	*Penalizes cells already targeted by other robots*
east_bonus_weight_regular	*0.567*	*Rewards eastward assignment for green/yellow robots*
east_bonus_weight_red	*0.414*	*Rewards eastward assignment for red robots*
scarcity_le2_bonus	*0.885*	*Bonus when ≤2 items of a color remain*
scarcity_eq1_bonus	*2.074*	*Bonus when exactly 1 item of a color remains*

Table A1 — Final calibrated uncertainty-aware scoring weights.

These weights are hardcoded as defaults in the orchestrator. They can be overridden via CLI arguments to run_step_impact_experiments.py for ablation or further calibration.